



WORKFLOW GUIDE / STARTER KIT

Motion as Code

Make motion graphics from code, driven by a voiceover,
without opening After Effects. Unzip, paste into your
project, run the demo prompt.

1920x1080 / 60 fps / TypeScript + three.js / bun + Vite

Visual engine: pdoom-video by mexicat (MIT). See the last page.

What you are getting

This kit is a complete, working example of the **prompt, code, render, video** workflow. It produces a 92.6 second, 1920x1080, 60 fps explainer whose script is: *"What if I told you you can create motion graphics like this without even opening After Effects?"*

Nothing in it is keyframed by hand. Each scene (called a **plate**) is a TypeScript file that draws every frame as a function of time. Word-level timings of the voiceover tell each plate when to animate, so the picture is locked to the voice.

Because it is all code, you can ask Claude to change the animation (a colour, a speed, a transition, a whole new plate) and re-render, instead of moving hundreds of keyframes.



Every frame is a pure function of time, so the browser preview and the exported MP4 are identical.

Re-timing: change the voiceover, re-run step 2, and the scenes find their words by content.

Install in five minutes

What to put in the zip. Include these folders and files. Leave out out/ (renders, over 2 GB) and app/node_modules (rebuilt by the install step).

Include	Why
app/	The renderer: engine, scenes, timeline, fonts, render script.
audio/	The voiceover and the 28 sound effects.
data/	Pre-computed word timings and audio analysis (so you can preview straight away).
analysis/	Alignment and SFX-mix scripts. The 95 MB ONNX model in analysis/models is only needed to re-align a new voiceover.
README.md, LICENSE.pdoom-engine, .claude/	Project guide, engine licence, one-click preview config.

Paste into your project. Unzip, then copy the contents into the folder you work in (your Claude Code project folder, or add the folder to the project's files). Keep the folder structure exactly as it is, because the scripts use relative paths.

Requirements.

- **bun** (bun.sh): installs packages and runs the scripts.
- **Google Chrome**: the renderer drives it headlessly to draw frames.
- **ffmpeg** with libx264: turns frames into MP4 and mixes sound.
- **Python + uv** (only to re-align a new voiceover or rebuild the SFX mix). Run it as `python -m uv`.

First run, from the project root:

```
cd app
bun install
bunx vite
```

Open `http://localhost:5173`. Press space to play (the voiceover plays with it). If a render later cannot find Chrome, see [Troubleshooting](#) on the last page.

03 / WORKFLOW

Step by step

1. Preview (live)

The preview is the fast loop. Edit a scene file, the page hot-reloads, scrub to the moment you changed.

Key	Action
space	play / pause
left / right	seek 1 s (5 s with shift)
, and .	step one frame
[and]	previous / next plate
l	loop the current plate
h	hide the UI

Add `?t=35` to the URL to start at 35 s.

2. Check stills while editing

```
cd app
bun scripts/render.ts stills --t 12.5,40.2 --only code --out ../out/wip
```

Renders single frames (here the code plate at 12.5 s and 40.2 s) so you can inspect a detail without rendering the whole video.

3. Render the video

```
cd app
bun run render
```

That is the npm script from `package.json`: automatic motion blur (4 to 12 sub-frames), shutter 0.5, CRF 17, written to `out/motion-as-code.mp4`. Useful flags:

Flag	Effect
<code>--samples 4</code>	About 4x faster; fast moves show stepped copies.

Flag	Effect
--samples auto	More sub-frames where motion is fast (default script).
--scale 2	4K, 3840x2160.

4. Add sound effects

There are 28 sound effects in `audio/sfx` and a cue sheet of about 500 cues in `analysis/sfx_mix.py`, derived from the same word timings. Effects duck up to 7 dB under the voice and the mix is normalised to -14 LUFS. No re-render is needed: rebuild the mix, then copy the picture and add the audio.

```
python -m uv run --no-project --with numpy python analysis/sfx_mix.py
cd out
ffmpeg -i motion-as-code.mp4 -i mix.wav -map 0:v -map 1:a \
  -c:v copy -c:a aac -b:a 320k -shortest motion-as-code_sfx.mp4
```

To make an effect louder or quieter, change its db value in the cue sheet. To move one, change its time expression.

5. Swap the voiceover

- Replace `audio/voiceover.mp3`.
- Edit `SCRIPT` (and `SPOKEN` for numbers and acronyms) in `analysis/align_vo.py`.
- Re-run the alignment and audio analysis:

```
python -m uv run --no-project --with onnxruntime --with numpy python analysis/align_vo.py
python -m uv run --no-project --with numpy python analysis/audio_vo.py
```

The plates find their words by content (`lineOf` and `wordOf`). If you keep the same script, every animation re-times itself to the new read.

04 / REFERENCE

How the project fits together

Path	Role
<code>audio/voiceover.mp3</code>	The voiceover (made with ElevenLabs).
<code>data/lyrics.json</code>	Word-level timings. Made by <code>align_vo.py</code> with CTC forced alignment (<code>wav2vec2</code> , quantized ONNX), refined against the audio's silences.
<code>data/audio.json</code>	Voice loudness envelopes, word onsets and a nominal beat grid. Made by <code>audio_vo.py</code> .
<code>app/src/engine/</code>	The pdoom-video engine (WebGL, post-processing, type, strokes). Leave this alone.
<code>app/src/scenes/_vo.ts</code>	Shared toolkit: 2D camera, the plotter pen, world-space karaoke words, graph paper.
<code>app/src/scenes/*.ts</code>	The nine plates.
<code>app/src/timeline.ts</code>	When each plate plays. Cuts sit in the pause just before each line.
<code>app/scripts/render.ts</code>	Offline renderer: headless Chrome to raw frames to ffmpeg.

The nine plates

Plate	Time (s)	Script
hook	0.0 - 9.9	What if I told you... / And no, not an After Effects MCP
model	9.9 - 19.2	This is Claude Opus 5.5 / Instead of controlling After Effects...
prompt	19.2 - 29.7	For example: "Create a 15-second cinematic..."
crazy	29.7 - 35.4	And this is where it gets crazy / not an MP4 directly
code	35.4 - 48.0	It writes the animation itself as code... defined mathematically over time
frames	48.0 - 52.1	Rendered frame by frame and turned into a video
pipeline	52.1 - 62.6	After Effects, layers, keyframes... vs Prompt, code, render, video
edits	62.6 - 80.6	Procedural / the four commands / hundreds of keyframes
verdict	80.6 - 92.6	Does this replace After Effects? Not really... pretty insane (loops to frame 0)

Writing or changing a plate

A plate is a class that extends `Scene` (`app/src/engine/scene.ts`) and exports it as default. The rule that matters: **everything must be a deterministic function of time**. The engine calls `render(f, out)` once per frame with:

- `f.t` song time, `f.lt` time since the plate started, `f.p` progress 0 to 1 through the plate.
- `f.a` audio features at that moment (voice envelope, hit pulses).
- `f.seeked` true after a scrub. Stateful plates reset on it.

To add a plate: create `app/src/scenes/yourplate.ts`, then add one `E('yourplate', 'yourplate', start, end)` line to `timeline.ts`. Start times come from the script, for example `cut('This is Claude')` cuts 0.18 s before that line's first word.

The example prompt shown inside the video (plate *prompt*) draws its token breakdowns from `app/src/scenes/prompt-data.ts`. Edit that file to change the words and the next-token candidates it displays.

Copy this into Claude

After the folder is pasted into your project, open Claude in that folder and send the prompt below. It verifies the install, makes one visible edit, then walks you into using your own voiceover. Nothing is rendered in full without your say-so.

I've added the "Motion as Code" project to this folder. Read README.md first, then app/src/timeline.ts and app/src/engine/scene.ts so you understand how plates and timing work.

Step 1 - verify:

cd app, run bun install, start the preview with bunx vite, and confirm the video plays at localhost:5173 with no console errors. Tell me what you see.

Step 2 - one small change to prove it works:

In the 'verdict' plate (app/src/scenes/verdict.ts), change the final full-frame "pretty insane." slam so the words hold for 0.3 s longer before the implode.

Keep it a pure function of time and keep the loop back to frame 0 intact.

Render a still at the new moment with:

bun scripts/render.ts stills --t 89.5 --only verdict --out ../out/wip
and show it to me.

Step 3 - make it mine:

Ask me for my voiceover (mp3) and script. Then update analysis/align_vo.py (SCRIPT and SPOKEN), re-run the alignment and audio analysis, and tell me which plate cuts in timeline.ts need new anchor phrases.

Rules: do not edit app/src/engine/. Do not run the full 4K render without asking. Explain each change in one or two sentences before you make it.

Ideas for your own follow-up prompts

- "Make the transition into the *code* plate smoother and slower."
- "Change the accent colour from signal orange to electric blue everywhere."
- "Slow the *pipeline* plate down by 20% without changing the voiceover timing."
- "Add a new plate between *frames* and *pipeline* that shows a loading bar."
- "Render the whole video with --samples 4 as a fast draft."

The example prompt used in the video itself reads: "Create a 15-second cinematic motion graphics sequence with kinetic typography, smooth shape transitions, 3D elements and seamless camera movement." You can paste it into Claude as a plain test of the idea.

Troubleshooting and credits

Problem	Try this
Blank page at localhost:5173	Run bun install inside app/ first, and check the terminal running vite for errors.
No sound in the preview	Press space to start playback; the voiceover is played by the page and the browser may block it until you interact. Check the tab is not muted.

Problem	Try this
Render cannot find Chrome	Install Google Chrome. The renderer launches the 'chrome' channel through Playwright; set the <code>BROWSER_CHANNEL</code> environment variable (for example to <code>msedge</code>) to use another installed browser.
ffmpeg not found / no libx264	Install a full ffmpeg build and make sure <code>ffmpeg -version</code> works in a new terminal.
Render is very slow	Use <code>--samples 4</code> for drafts and render single plates with the <code>stills</code> command.
Animation drifted after a new voiceover	Re-run both analysis scripts. If the wording changed, update the phrases in <code>cut (. . .)</code> calls in <code>timeline.ts</code> .
Scene module not found	The file name in <code>timeline.ts</code> must match a file in <code>app/src/scenes/</code> .

Credits and licence

The visual language is the engine and style of **pdoom-video** by mexicat (Giacomo Magnanini), MIT licence, included as `LICENSE.pdoom-engine`. That covers the palette (ink, bone, signal orange), the fonts (Archivo, IBM Plex Mono, Cormorant Garamond and the single-stroke plotter fonts), the spark motif, the graph-paper construction sheets, and the post-processing (bloom, halation, grain). Keep that licence file with the project. The nine plates themselves are new and were written for this voiceover.

Voiceover by ElevenLabs. Sound effects generated with ElevenLabs Sound Effects v2. Check the licence terms of your own ElevenLabs plan before publishing a video that uses your own generated audio.